Machine Learning od podstaw

Praktyczny przewodnik na bazie projektu
predykcji ceny samochodu i zużycia paliwa

Ten dokument wyjaśnia krok po kroku, jak działa uczenie maszynowe (Machine Learning) na przykładzie realnego zbioru danych o samochodach. Wszystkie koncepcje są zilustrowane kodem, który sam uruchamiałeś, oraz wynikami, które otrzymałeś.

Zawartość: dane i ich przygotowanie • eksploracja (EDA) • podział train/val/test • regresja liniowa • metryka RMSE • inżynieria cech • zmienne kategoryczne • regularyzacja • modele drzewiaste • porównanie modeli • interpretacja wyników.

1. Czym jest uczenie maszynowe?

Uczenie maszynowe to sposób, w którym komputer **uczy się z danych**, zamiast być programowany regułami ręcznie. Zamiast pisać "jeśli moc silnika > 300, to cena wysoka", pokazujemy modelowi tysiące przykładów (samochód + jego cena), a on sam odkrywa zależności.

Rodzaje uczenia:

- **Uczenie nadzorowane** — mamy dane wejściowe (X) i znane odpowiedzi (y). Model uczy się mapowania X → y. Nasz projekt jest właśnie taki.
- **Regresja** — przewidujemy liczbę (np. cena, MPG). To robimy w tym projekcie.
- **Klasyfikacja** — przewidujemy kategorię (np. czy auto jest drogie: tak/nie).
- **Uczenie nienadzorowane** — brak odpowiedzi, szukamy struktury (np. grupowanie podobnych aut).

W naszym projekcie: **X** = cechy samochodu (moc, marka, rok...), **y** = cena (MSRP). Model uczy się przewidywać cenę na podstawie cech.

2. Dane i ich przygotowanie

Wszystko zaczyna się od danych. Nasz plik `data.csv` zawiera ok. 12 000 samochodów, każdy opisany 16 kolumnami:

Kolumna	Znaczenie	Typ
Make, Model	marka, model	tekst
Year	rok produkcji	liczba
Engine HP	moc silnika (konie mechaniczne)	liczba
Engine Cylinders	liczba cylindrów	liczba
highway MPG, city mpg	zużycie paliwa	liczba
Popularity	popularność modelu	liczba
MSRP	cena (nasz cel!)	liczba

Normalizacja nazw kolumn

Nazwy kolumn mają spacje i wielkie litery. Zamieniamy je na małe litery i podkreślenia, żeby łatwiej się do nich odwoływać:

```
df.columns = df.columns.str.lower().str.replace(' ', '_')
# 'Engine HP'  ->  'engine_hp'
```

To samo robimy z wartościami tekstowymi (marki, typy paliwa), żeby np. 'BMW' i 'bmw' były traktowane identycznie.

3. Eksploracja danych (EDA)

EDA (Exploratory Data Analysis) to poznanie danych przed modelowaniem. Sprawdzamy rozkłady, braki i dziwne wartości.

Długi ogon w rozkładzie ceny

Ceny samochodów mają rozkład z **długim ogonem**: większość aut jest tania, ale kilka jest ekstremalnie drogich (Ferrari, Maybach). To psuje modele liniowe. Rozwiązanie: **logarytm**.

```
price_logs = np.log1p(df.msrp)   # log(1 + cena)
# log1p jest bezpieczne dla zera i ściska duże wartości
```

Po logarytmowaniu rozkład staje się bardziej symetryczny (dzwonowy), co bardzo pomaga modelom. Pamiętaj: przewidujemy log(cenę), a na końcu wracamy do złotówek przez `np.expm1()`.

Braki danych

Niektóre auta nie mają podanej mocy silnika. Zamiast usuwać wiersze, uzupełniamy braki zerem (proste i skuteczne w tym przypadku):

```
df.isnull().sum()          # ile braków w każdej kolumnie
df_num = df_num.fillna(0)  # uzupełnij braki zerem
```

4. Podział danych: train / val / test

To jeden z najważniejszych kroków. **Nigdy nie oceniamy modelu na danych, na których się uczył** — to jak sprawdzanie ucznia na tych samych pytaniach, które widział. Dzielimy dane na trzy części:

Zbiór	Udział	Do czego służy
train	60%	model uczy się na tych danych
val (walidacja)	20%	wybieramy najlepszy model / parametry
test	20%	ostateczna, uczciwa ocena (dotykamy raz!)

```
n = len(df)
n_val = int(n * 0.2)
n_test = int(n * 0.2)
n_train = n - n_val - n_test

idx = np.arange(n)
np.random.seed(2)          # powtarzalność wyników
np.random.shuffle(idx)     # losowe przetasowanie

df_train = df.iloc[idx[:n_train]]
df_val   = df.iloc[idx[n_train:n_train+n_val]]
df_test  = df.iloc[idx[n_train+n_val:]]
```

Dlaczego tasujemy? Bo dane w pliku są posortowane (np. wszystkie BMW razem). Bez tasowania model uczyłby się na jednych markach, a testowany był na innych — wynik byłby fałszywie zły.

5. Regresja liniowa — pierwszy model

Najprostszy model: przewidywana cena to suma ważona cech plus stała:

$$y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n$$

gdzie $\mathbf{w_0}$ to wyraz wolny (bias), a $\mathbf{w_1 \ldots w_n}$ to wagi (jak bardzo każda cecha wpływa na cenę). Uczenie = znalezienie takich wag, żeby predykcje były jak najbliżej prawdy.

Jak znaleźć wagi? Równanie normalne

Istnieje elegancki wzór matematyczny, który od razu daje optymalne wagi:

$$w = (X^T X)^{-1} X^T y$$

gdzie X to macierz cech (z dodatkową kolumną jedynek dla w_0), a y to wektor cen. To nie magia — to rozwiązanie problemu najmniejszych kwadratów.

```
def train_linear_regression(X, y):
    ones = np.ones(X.shape[0])
    X = np.column_stack([ones, X])   # dodaj kolumnę jedynek

    XTX = X.T.dot(X)
    XTX_inv = np.linalg.inv(XTX)     # macierz odwrotna
    w_full = XTX_inv.dot(X.T).dot(y)

    return w_full[0], w_full[1:]     # (w0, pozostałe wagi)
```

6. RMSE — jak mierzyć błąd?

Potrzebujemy liczby, która powie nam "jak bardzo model się myli". Najczęściej używamy **RMSE** (Root Mean Squared Error):

> RMSE = √(średnia($(y - y_pred)^2$))

Krok po kroku:

- Policz różnicę między prawdą a predykcją dla każdego auta.
- Podnieś każdą różnicę do kwadratu (żeby minusy nie znikały).
- Uśrednij te kwadraty (MSE).
- Wyciągnij pierwiastek (RMSE) — wraca do jednostek ceny.

```
def rmse(y, y_pred):
    se = (y - y_pred) ** 2
    mse = se.mean()
    return np.sqrt(mse)
```

> Im **niższe RMSE**, tym lepszy model. RMSE = 0 oznaczałoby idealne predykcje. W naszym projekcie (na log-cenie) RMSE ~0.11 dla modeli drzewiastych to bardzo dobry wynik.

7. Inżynieria cech (feature engineering)

Często sami tworzymy nowe, mądrzejsze cechy z istniejących. To jedna z najskuteczniejszych technik w ML.

Przykład 1: wiek samochodu

Rok produkcji (np. 2015) sam w sobie niewiele mówi. Ale **wiek** (2017 - rok) jest bezpośrednio związany z ceną:

```
df['age'] = 2017 - df['year']
```

Przykład 2: zmienne kategoryczne (one-hot encoding)

Model nie rozumie tekstu jak "bmw" czy "sedan". Zamieniamy każdą kategorię na kolumnę 0/1:

```
df['make_bmw'] = (df['make'] == 'bmw').astype(int)
# BMW -> 1, inne -> 0
```

Dla każdej ważnej kategorii tworzymy osobną kolumnę. To się nazywa **one-hot encoding**. Bierzemy tylko najczęstsze wartości (np. top 5 marek), żeby nie tworzyć tysięcy kolumn.

8. Regularyzacja — lekarstwo na niestabilność

Czasem macierz X^TX jest **osobliwa** (nie da się jej odwrócić), bo cechy są silnie skorelowane. Wtedy wagi eksplodują do absurdalnych wartości. Widzieliśmy to w wynikach:

```
r=0.0   ->  w0 = -292975507   RMSE = 113.55   (katastrofa!)
r=1e-05 ->  w0 = 7.88         RMSE = 0.46     (świetnie!)
```

Rozwiązanie: **regularyzacja**. Dodajemy małą wartość do przekątnej macierzy, co czyni ją odwracalną:

$$X^TX \rightarrow X^TX + r \cdot I$$

```
def train_linear_regression_reg(X, y, r=0.001):
    ones = np.ones(X.shape[0])
    X = np.column_stack([ones, X])

    XTX = X.T.dot(X)
    XTX = XTX + r * np.eye(XTX.shape[0])   # <-- regularyzacja

    XTX_inv = np.linalg.inv(XTX)
    w_full = XTX_inv.dot(X.T).dot(y)
    return w_full[0], w_full[1:]
```

r to parametr, który dostrajamy. Za mały → niestabilność. Za duży → model traci zdolność uczenia. Szukamy złotego środka na zbiorze walidacyjnym.

9. Modele drzewiaste — mocniejsze niż liniowe

Regresja liniowa zakłada, że zależności są **liniowe**. Ale cena samochodu zależy od cech w sposób nieliniowy (np. moc silnika podnosi cenę tylko dla marek premium). Tu wchodzą **drzewa decyzyjne**.

Jak działa drzewo?

Drzewo zadaje serię pytań: "czy moc > 200?", "czy wiek < 5?" i dzieli dane na grupy. Każda grupa dostaje swoją przewidywaną cenę. To naturalnie wychwytuje nieliniowości i interakcje cech.

Ensemble — siła w liczbie

Pojedyncze drzewo jest słabe. Ale **wiele drzew razem** (ensemble) daje świetne wyniki:

- **Random Forest** — buduje setki drzew na losowych podzbiorach danych i uśrednia ich predykcje.
- **Gradient Boosting** — buduje drzewa po kolei, każde poprawia błędy poprzedniego.
- **Hist Gradient Boosting** — szybsza wersja boosting, świetna dla dużych zbiorów. Nasz zwycięzca!

```
from sklearn.ensemble import HistGradientBoostingRegressor
```

```
model = HistGradientBoostingRegressor(
    max_iter=300, learning_rate=0.1, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

10. Porównanie modeli — wyniki

Uruchomiliśmy cztery modele na tych samych danych. Oto wyniki (RMSE na log-cenie, niżej = lepiej):

Model	RMSE (walidacja)	RMSE (test)
Ridge (regresja liniowa)	0.4608	0.4605
Random Forest	0.1134	0.1203
Gradient Boosting	0.1508	0.1550
Hist Gradient Boosting	0.1112	0.1156

Wnioski:

- Modele drzewiaste są **~4x lepsze** od liniowego (0.11 vs 0.46).
- **Hist Gradient Boosting** wygrywa — najniższy błąd.
- Regresja liniowa jest najsłabsza, bo nie wychwytuje nieliniowości.

Ważne: RMSE na walidacji i teście są **zbliżone** (np. 0.1112 vs 0.1156). To znaczy, że model **nie przeucza się** — dobrze generalizuje na nowe dane.

11. Interpretacja wyników

Co wpływa na cenę? (ważność cech)

Random Forest potrafi powiedzieć, które cechy są najważniejsze. W naszym modelu:

- **age (wiek)** — dominująca cecha (~75%). Starsze auto = tańsze.
- **engine_hp (moc)** — druga najważniejsza (~20%).
- Pozostałe cechy mają marginalne znaczenie.

Nakładka predykcji i prawdy

Gdy nałożymy rozkład przewidywanych cen na prawdziwe, histogramy **pokrywają się niemal idealnie**. To dowód, że model nie ma systematycznego obciążenia (bias).

Gdzie model się myli?

- Największe błędy przy **skrajnych cenach** (bardzo tanich i drogich) — jest ich mało w danych.
- Auta elektryczne/hybrydowe (skrajne MPG) są trudne do przewidzenia.
- W głównym zakresie cen model jest bardzo dokładny.

12. Drugi projekt: predykcja zużycia paliwa

Ten sam warsztat zastosowaliśmy do przewidywania MPG (zużycia paliwa). Zmieniliśmy tylko cel (y) z ceny na MPG:

Cel	RMSE (walidacja)	RMSE (test)
highway_mpg	1.8052	1.5207
city_mpg	1.4908	1.3434

RMSE ~1.5 MPG to ok. 5-7% typowego błędu — bardzo dobry wynik. City MPG jest łatwiejsze do przewidzenia niż highway.

13. Podsumowanie — cały proces ML

Powtórzmy pełny przepływ pracy, który wykonaliśmy:

Krok	Co robimy	Dlaczego
1. Dane	wczytanie, normalizacja	spójny format
2. EDA	rozkłady, braki	poznać dane
3. Target	log1p(cena)	symetryczny rozkład
4. Podział	train/val/test	uczciwa ocena
5. Model	regresja / drzewa	nauczyć zależności
6. Metryka	RMSE	zmierzyć błąd
7. Cechy	age, one-hot	lepsze sygnały
8. Regularyzacja	r * I	stabilność
9. Tuning	wybór r / modelu	najlepszy wynik
10. Ocena	RMSE na teście	ostateczna prawda

Złote zasady ML

- **Nigdy** nie oceniaj modelu na danych treningowych.
- Zawsze trzymaj zbiór testowy w tajemnicy do samego końca.
- Zacznij od prostego modelu (baseline), potem komplikuj.
- Mierz błąd na walidacji, wybieraj najlepszy wariant.
- Dbaj o to, by walidacja i test dawały podobne wyniki (brak przeuczenia).
- Interpretuj wyniki — sprawdzaj, czy mają sens biznesowy.

Gratulacje! Przeszedłeś przez kompletny projekt uczenia maszynowego — od surowych danych do działającego modelu predykcyjnego. Te same kroki stosuje się w każdym projekcie ML, niezależnie od dziedziny.